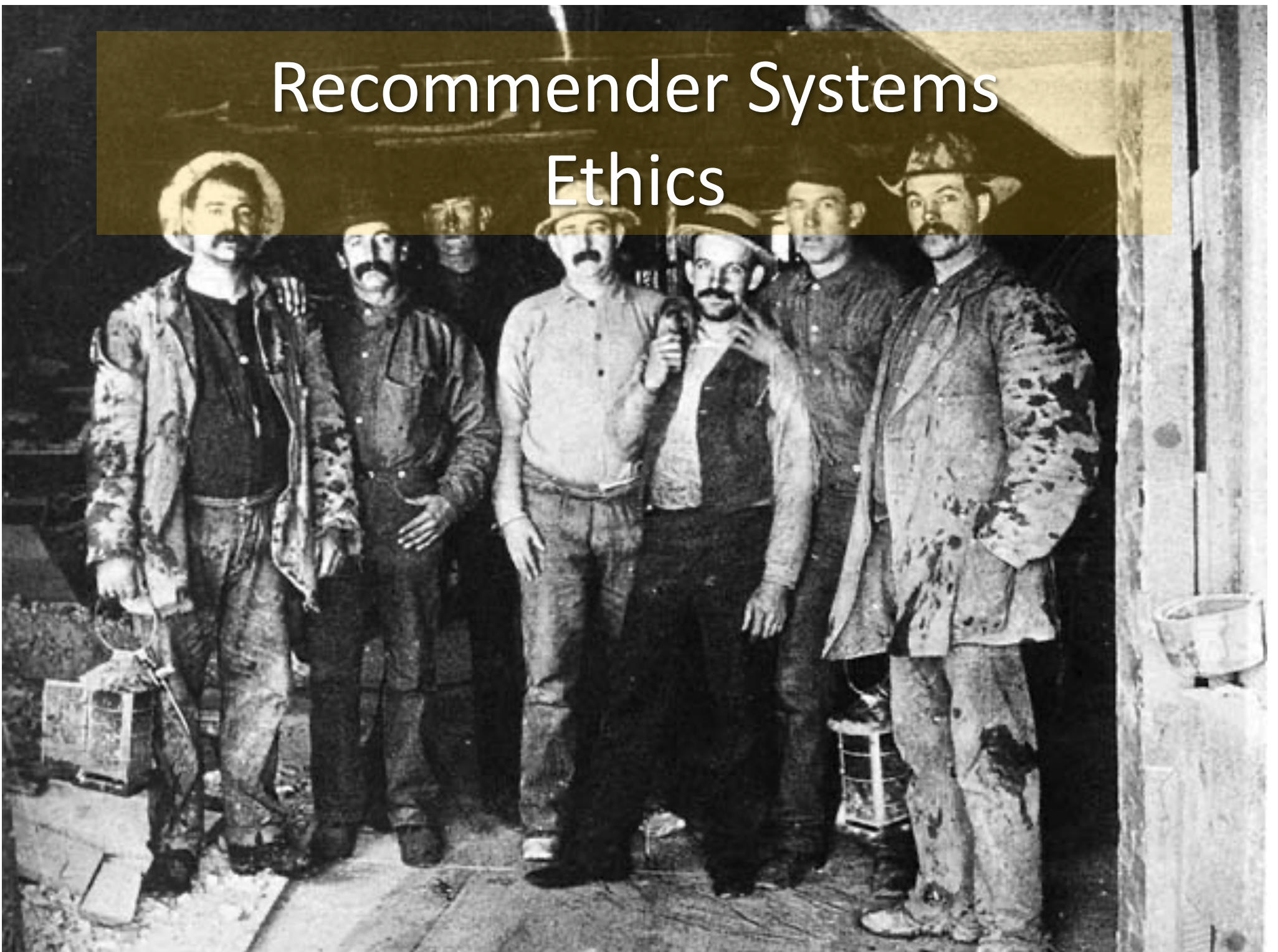


Recommender Systems Ethics



Recommender system

Recommender systems are a subclass of information filtering systems that seek to predict the 'rating' or 'preference' that users would give to an item (such as music, books, or movies) or social element (e.g. people or groups) they had not yet considered, using a model built from the characteristics of an item or the user's social environment (Ricci, Rokach and Shapira)

What kind of data can we have?

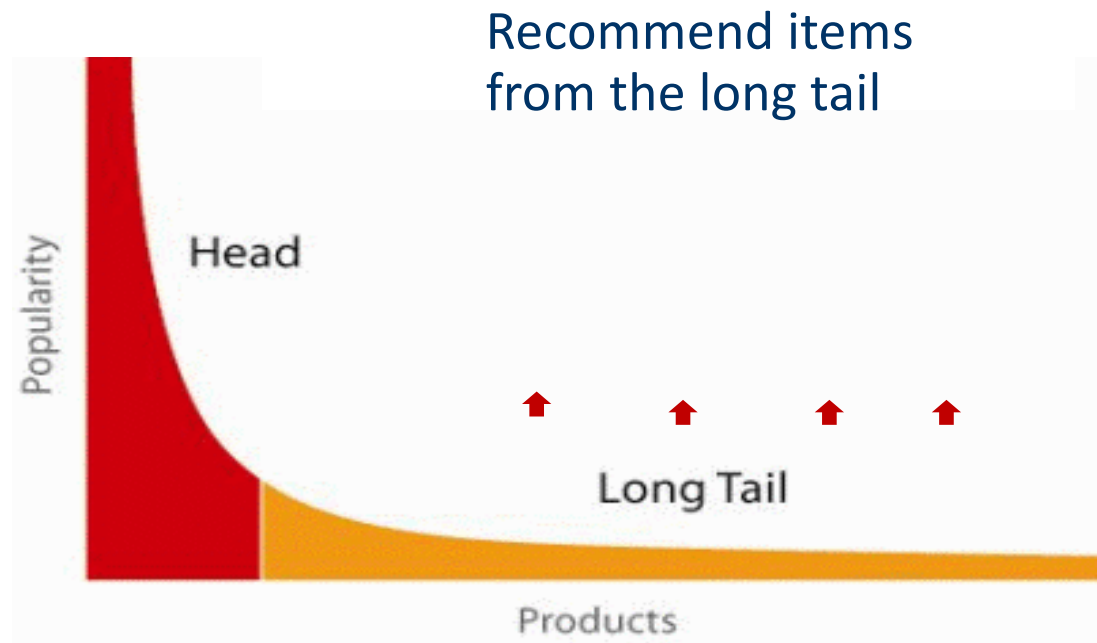
- Data about users
- Data about items
- Data about user-item interactions
 - Actions (you bought this)
 - Ratings (you bought this and rated it as 9/10)

A possible data matrix

					I1	I2	I3	I4	...	In
					Title1	Title2	Title3	Title4	...	Titlen
					Actor1	Actor2	Actor3	Actor4	...	Actorn
					...	...	...	...	...	...
					Desc1	Desc2	Desc3	Desc4	...	Descrn
U1	Sex1	FavX1	...	Desc1	3		2	2		
U2	Sex2	FavX2	...	Desc2	5	4	4			
U3	Sex3	FavX3	...	Desc3	1	1	1	1	1	1
U4	Sex4	FavX4	...	Desc4				5		3
...	...	...	...	...	3		2			
Um	Sexm	FavXm	...	Descm				5		

So how easy is this problem?

And can we exploit this matrix?



- "Recommend widely unknown items that users might actually like!"
- 20% of items accumulate 74% of all positive ratings
- Items rated > 3 in MovieLens 100K dataset

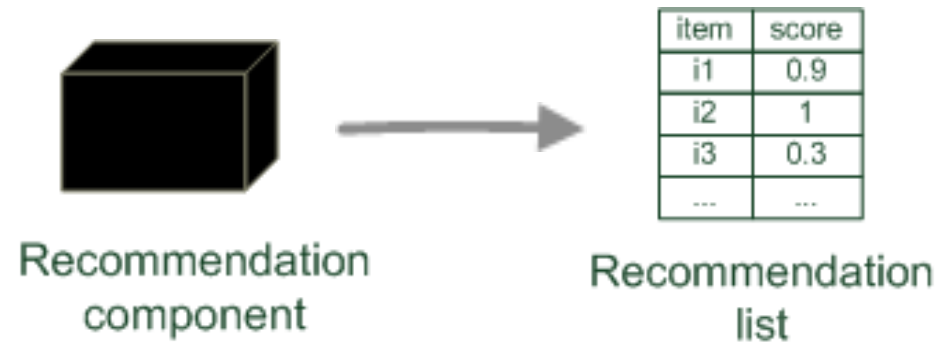
- Next deck of slides largely from Jannach, D., Zanker, M., Felfernig, A., & Friedrich, G. (2011). Recommender systems, an introduction. Cambridge University Press.

Purpose and success criteria

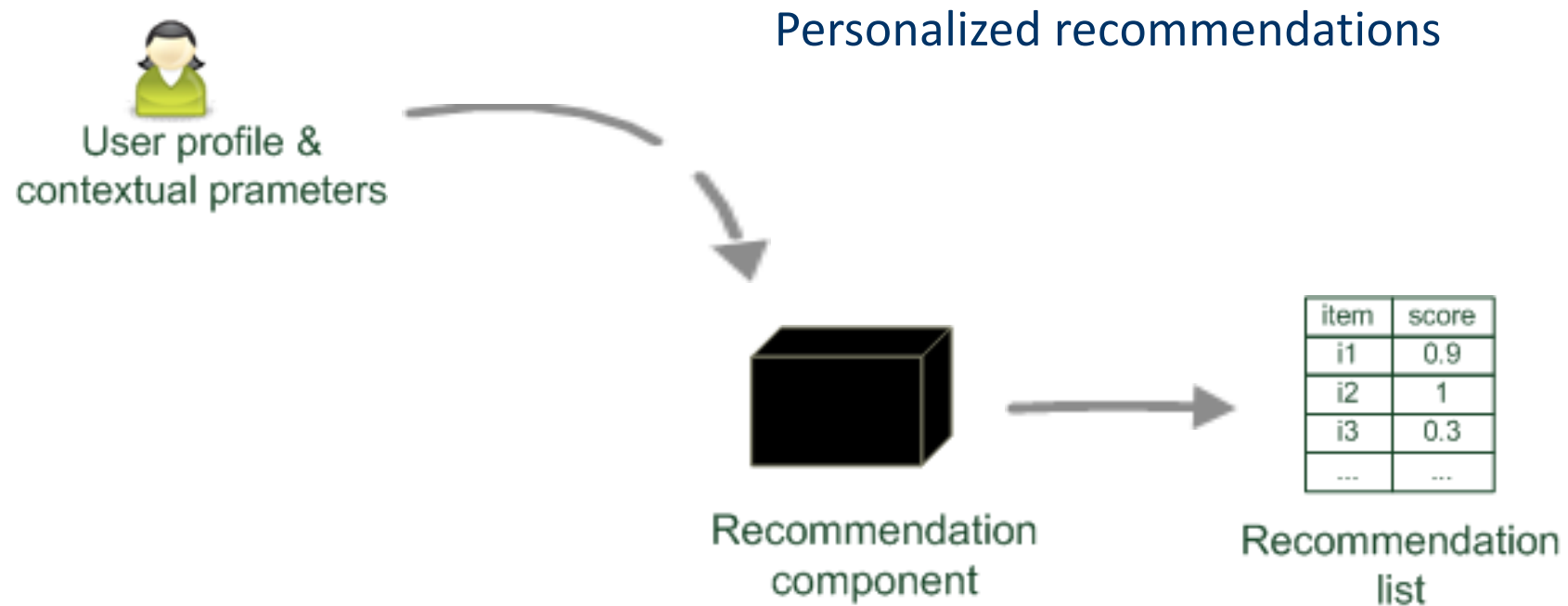
- Prediction perspective
 - Predict to what degree users like an item
 - Most popular evaluation scenario in research
- Interaction perspective
 - Give users a "good feeling"
 - Educate users about the product domain
 - Convince/persuade users - explain
- Finally, conversion perspective
 - Commercial situations
 - Increase "hit", "clickthrough", "lookers to bookers" rates
 - Optimize sales margins and profit

Paradigms of recommender systems

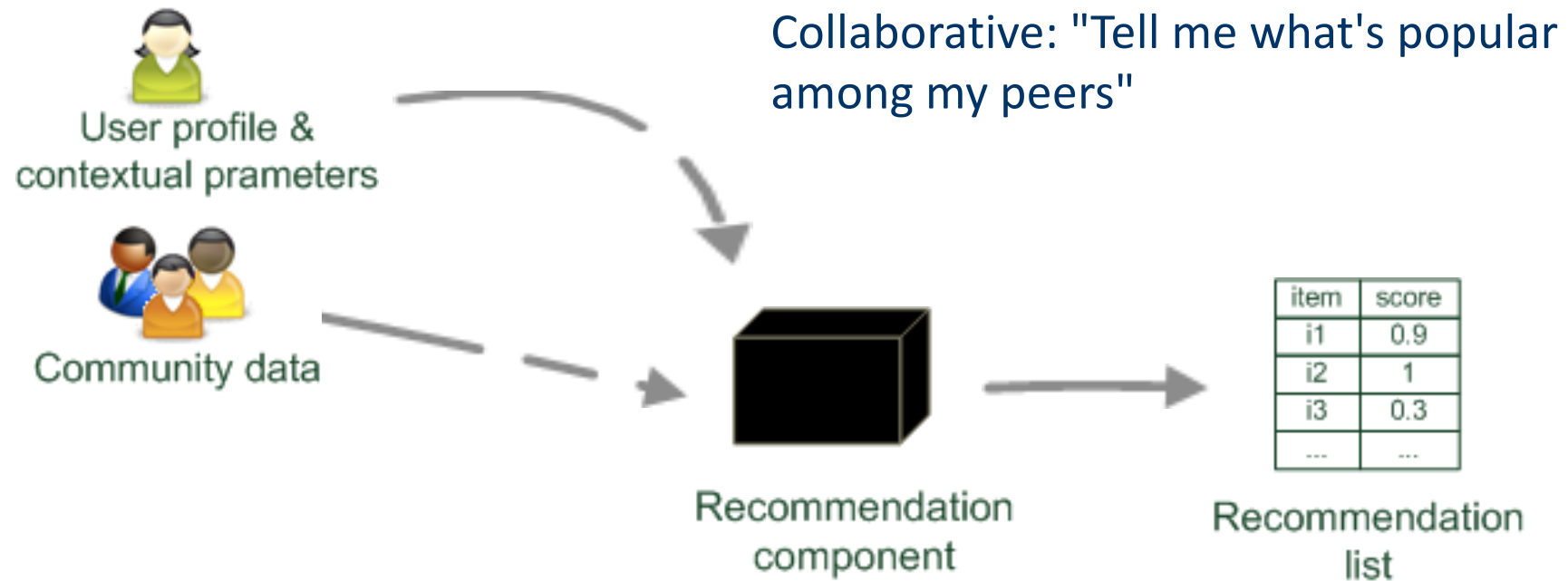
Recommender systems reduce information overload by estimating relevance



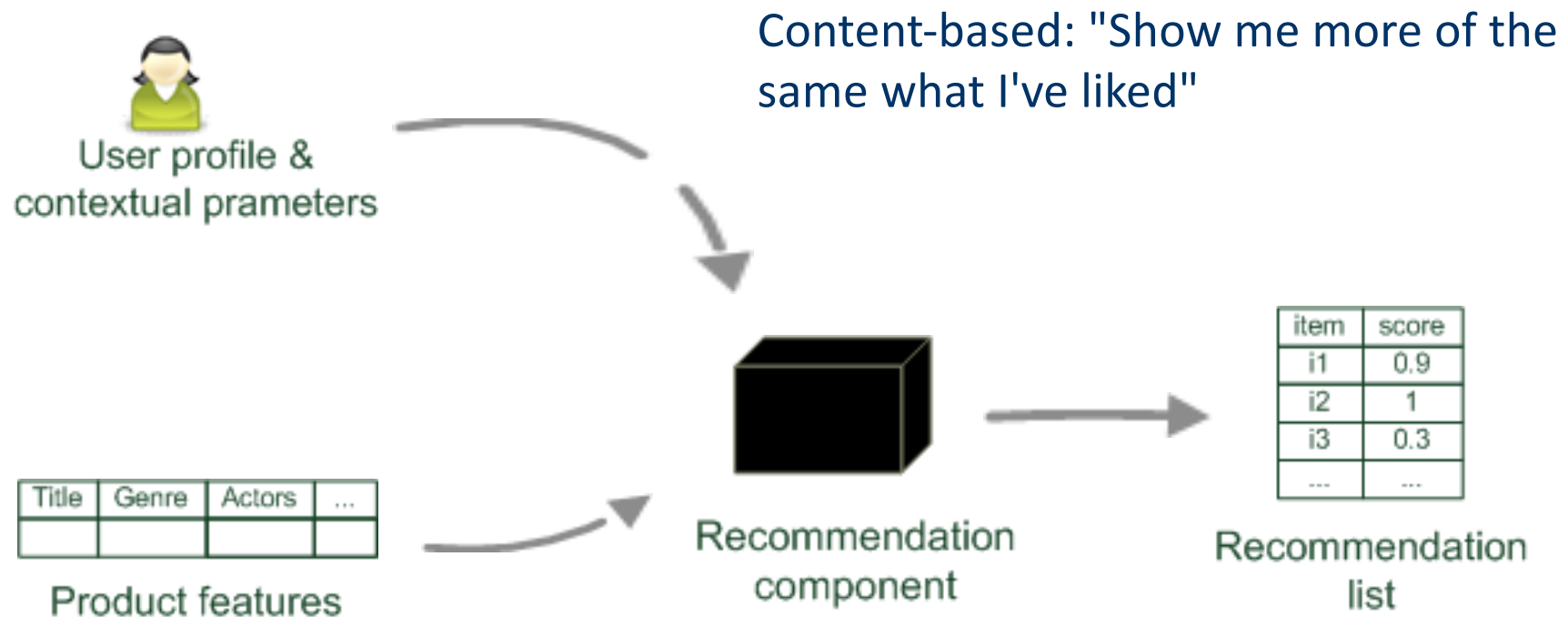
Paradigms of recommender systems



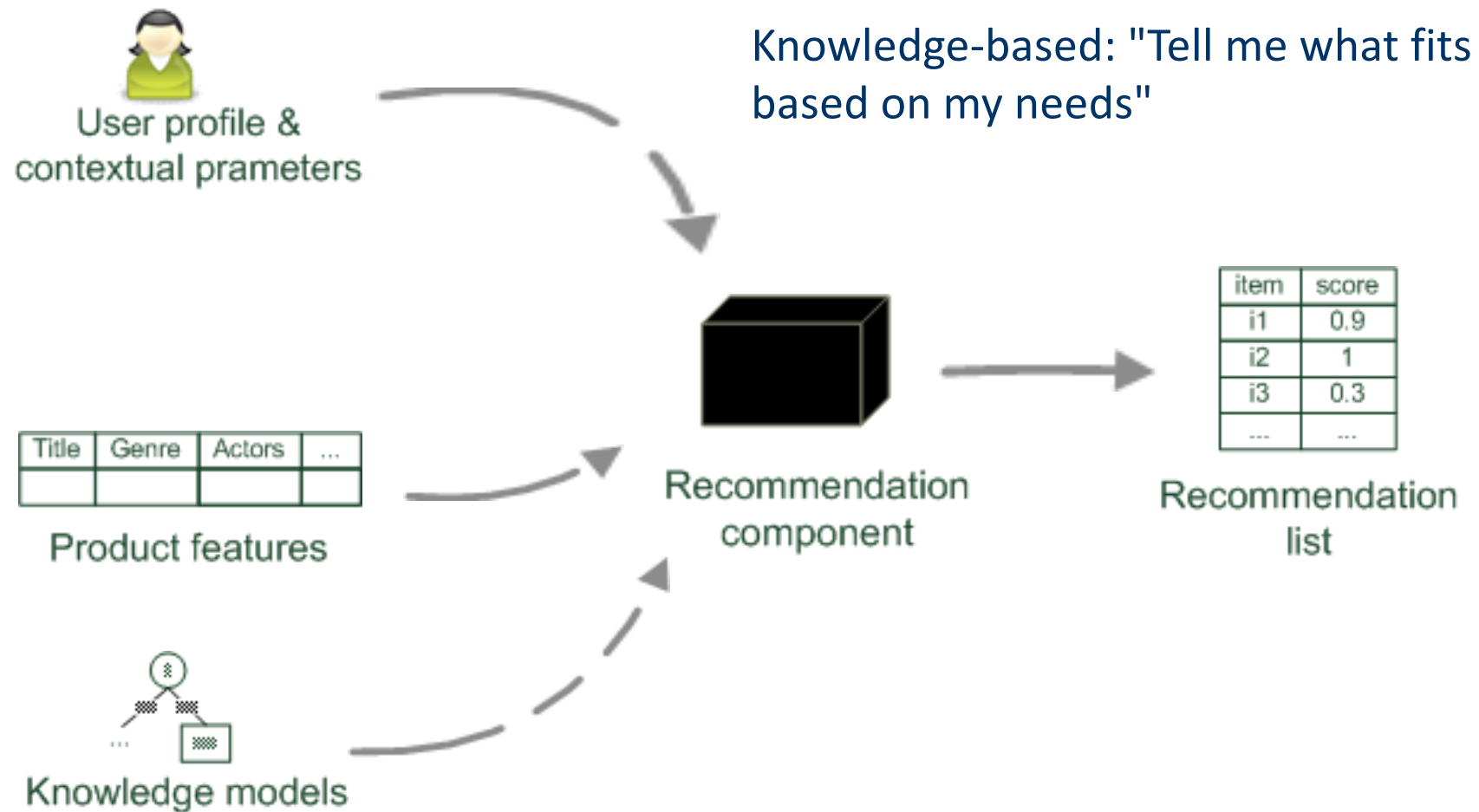
Paradigms of recommender systems



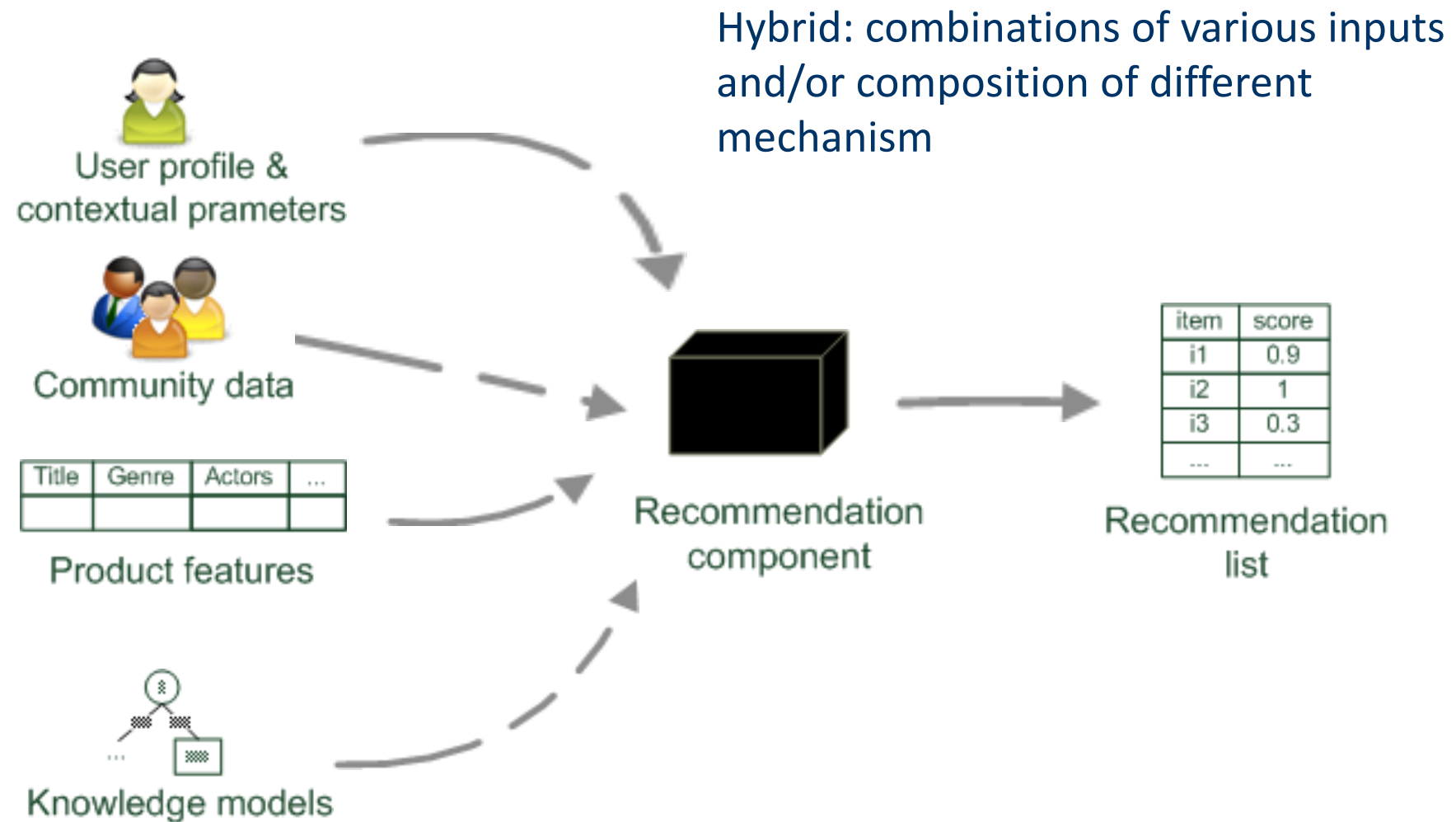
Paradigms of recommender systems



Paradigms of recommender systems



Paradigms of recommender systems



Filtering Approaches

- Collaborative filtering and content-based filtering most popular
- Let's zoom into these approaches a bit more
 - Note that we look into the *basics*

Collaborative filtering

- The most prominent approach to generate recommendations
 - used by large, commercial e-commerce sites
 - well-understood, various algorithms and variations exist
 - many domains (books, movies, social media posts)
- Approach
 - use the "wisdom of the crowd" to recommend items
- Basic assumption and idea
 - Users give ratings to catalog items (implicitly or explicitly)
 - Customers who had similar tastes in the past, will have similar tastes in the future

A Pure CF approach

- Input
 - Only a matrix of given user–item ratings

	Item1	Item2	Item3	Item4	Item5
Alice	5	3	4	4	?
User1	3	1	2	3	3
User2	4	3	4	3	5
User3	3	3	1	5	4
User4	1	5	5	2	1

- Output types
 - A (numerical) prediction indicating to what degree the current user will like or dislike a certain item
 - A top-N list of recommended items

K-Nearest Neighbors

- The basic idea:
 - Given a user (e.g. Alice) and an item Alice has not seen before
 - Find a set of k users that are like Alice but have rated the item
 - Use e.g. the average value of their rating to predict the rating of Alice
 - Do this for all items Alice has not seen
 - Basic assumptions:
 - Similarity based on past ratings reflects current/future similarity
 - User preferences remain stable over time

K-Nearest Neighbors

- How do we measure similarity between users?
- How do we select k ?
- How do we aggregate the k rating values we get to create the rating for the current user?

K-Nearest Neighbors

- How do we measure similarity between users?
 - Pearson correlation

a, b : users

$r_{a,p}$: rating of user a for item p

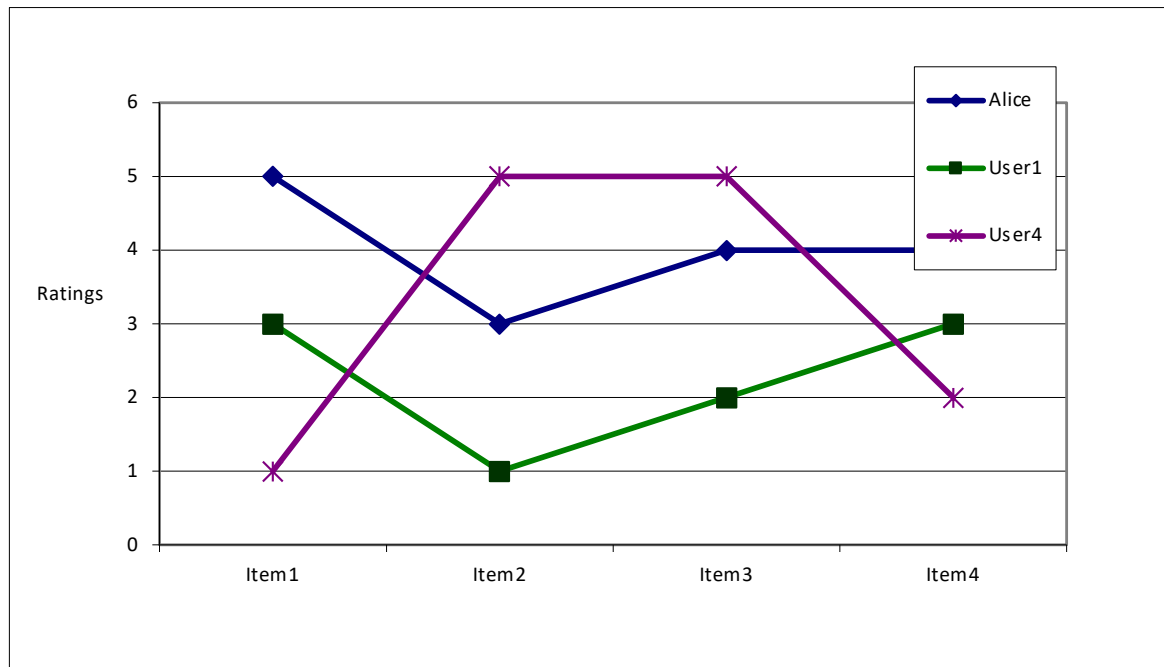
P : set of items, rated both by a and b

- Possible similarity values between -1 and 1

$$\text{sim}(a, b) = \frac{\sum_{p \in P} (r_{a,p} - \bar{r}_a)(r_{b,p} - \bar{r}_b)}{\sqrt{\sum_{p \in P} (r_{a,p} - \bar{r}_a)^2} \sqrt{\sum_{p \in P} (r_{b,p} - \bar{r}_b)^2}}$$

K-Nearest Neighbors

- How do we measure similarity between users?
 - Pearson correlation



K-Nearest Neighbors

- How do we select k ?
 - You guessed it, trial and error...
- How do we aggregate the k rating values we get to create the rating for the current user?

- Common approach:

$$pred(a, p) = \bar{r}_a + \frac{\sum_{b \in B} sim(a, b) * (r_{b,p} - \bar{r}_b)}{\sum_{b \in B} sim(a, b)}$$

for a 's nearest neighbors B

- Will this approach scale to Netflix size?
 - Nope, computation is done at runtime...

Item-based similarity

- Basic idea:
 - Use the similarity between items (and not users) to make predictions
- Example:
 - Look for items that are similar to Item5
 - Take Alice's ratings for these items to predict the rating for Item5

	Item1	Item2	Item3	Item4	Item5
Alice	5	3	4	4	?
User1	3	1	2	3	3
User2	4	3	4	3	5
User3	3	3	1	5	4
User4	1	5	5	2	1

Item-based similarity

- Let U be a set of users that have rated both items a and b :

$$\text{sim}(a,b) = \frac{\sum_{u \in U} (r_{u,a} - \bar{r}_u)(r_{u,b} - \bar{r}_u)}{\sqrt{\sum_{u \in U} (r_{u,a} - \bar{r}_u)^2} \sqrt{\sum_{u \in U} (r_{u,b} - \bar{r}_u)^2}}$$

- And we compute the prediction:

$$\text{pred}(u,p) = \frac{\sum_{i \in \text{ratedItem}(u)} \text{sim}(i,p) * r_{u,i}}{\sum_{i \in \text{ratedItem}(u)} \text{sim}(i,p)}$$

Singular Value Decomposition

- Let us consider our matrix M again (users x ratings)
- We can try to “describe” this matrix via Singular Value Decomposition:
 - Can decompose $M = U \times \Sigma \times V^T$
 - For example
 - $M = m \times n$ (m = users, n = items)
 - $U = m \times m$ (“represents” users) – *In Recommender systems mostly $m \times r$*
 - $\Sigma = m \times n$ – *In Recommender systems mostly $r \times r$ (these are non zero values)*
 - $V^T = n \times n$ (“represents” items) – *In Recommender systems mostly $r \times n$*
 - U are the left singular vectors, V the right singular vectors
 - Σ only has values in the diagonal
 - Are a sign for importance of elements (think of eigenvalues)
 - We select the most important elements
 - Note: we have a lot of missing values, therefore we use a variant of SVD that focuses on the “known” values

Singular Value Decomposition

- SVD: $M_k = U_k \times \Sigma_k \times V_k^T$

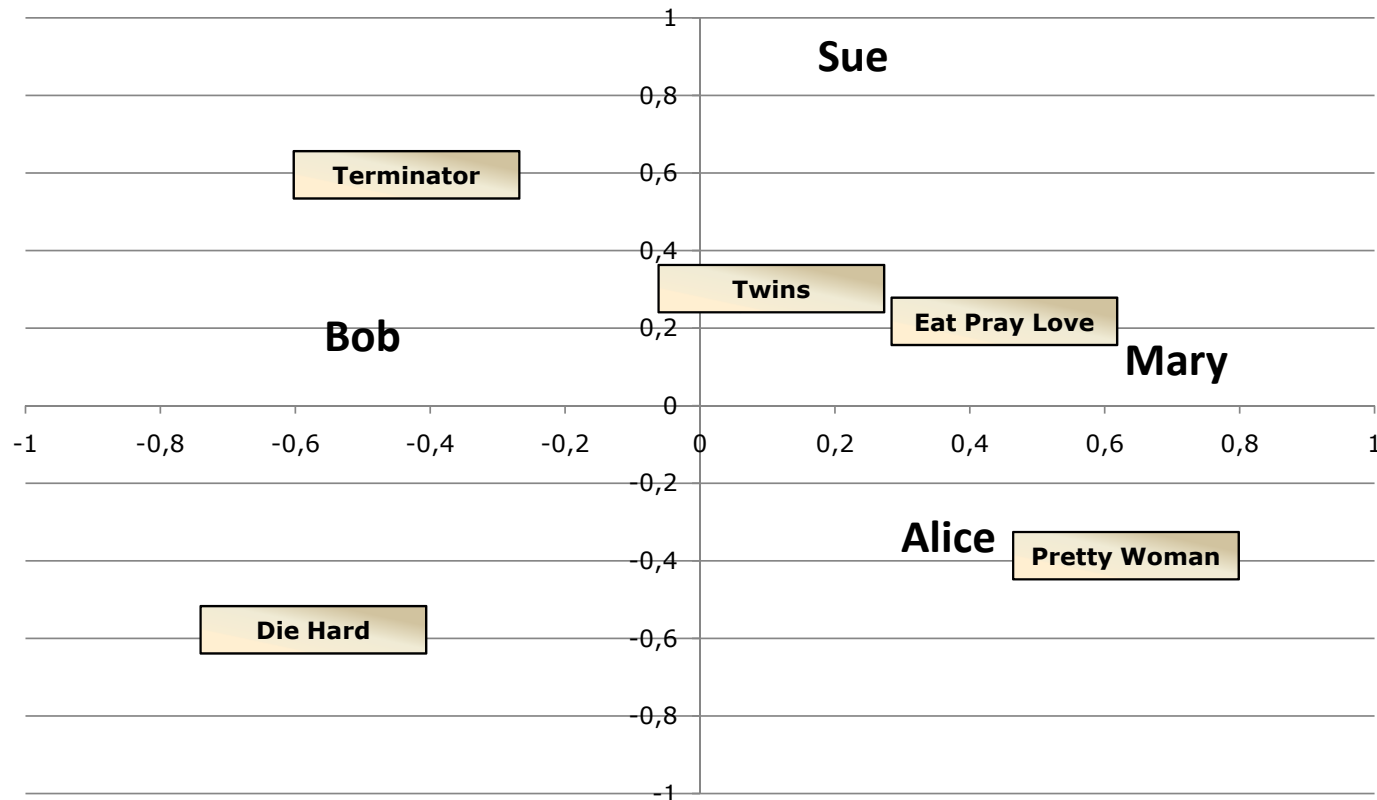
U_k	Dim1	Dim2
Alice	0.47	-0.30
Bob	-0.44	0.23
Mary	0.70	-0.06
Sue	0.31	0.93

V_k^T	Terminator	Die Hard	Twins	Eat Pray Love	Pretty Woman
Dim1	-0.44	-0.57	0.06	0.38	0.57
Dim2	0.58	-0.66	0.26	0.18	-0.36

Σ_k	Dim1	Dim2
Dim1	5.63	0
Dim2	0	3.23

- Prediction: $\hat{r}_{ui} = \bar{r}_u + U_k(\text{Alice}) \times \Sigma_k \times V_k^T(\text{EPL})$
 $= 3 + 0.84 = 3.84$

Can even visualize them....



Problem Collaborative Filtering

- **Cold start problem**
 - How to recommend new items? What to recommend to new users?
- **Straightforward approaches**
 - Ask/force users to rate a set of items
 - Use another method (e.g., content-based, demographic or simply non-personalized) in the initial phase
 - Default voting: assign default values to items that only one of the two users to be compared has rated (Breese et al. 1998)

Content-based recommendations

- While CF – methods do not require any information about the items,
 - it might be reasonable to exploit such information; and
 - recommend fantasy novels to people who liked fantasy novels in the past
- What do we need:
 - some information about the available items such as the genre ("content")
 - some sort of user profile describing what the user likes (the preferences)
- The task:
 - learn user preferences
 - locate/recommend items that are "similar" to the user preferences

Content-based recommendations

- What information do we have?
- Usually text based examples
- Let us focus on a bit of structured text, e.g. a fixed list of keywords

Content-based recommendations

- Items

Title	Genre	Author	Type	Price	Keywords
The Night of the Gun	Memoir	David Carr	Paperback	29.90	Press and journalism, drug addiction, personal memoirs, New York
The Lace Reader	Fiction, Mystery	Brunonia Barry	Hardcover	49.90	American contemporary fiction, detective, historical
Into the Fire	Romance, Suspense	Suzanne Brockmann	Hardcover	45.90	American fiction, murder, neo-Nazism

- User

Title	Genre	Author	Type	Price	Keywords
...	Fiction	Brunonia, Barry, Ken Follett	Paperback	25.65	Detective, murder, New York

- Compute similarity using e.g. DICE coefficient

$$\frac{2 \times |\text{keywords}(b_i) \cap \text{keywords}(b_j)|}{|\text{keywords}(b_i)| + |\text{keywords}(b_j)|}$$

Introducing to you...

- **Assignment from the past on this topic...**
- Topic:
 - Music recommendation
- Task:
 - Achieve as high scores as possible...
 - ...in separating items that people liked from those they haven't even listened to.

More details

#Users	#Items	#Ratings	#TrainRatings	#TestRatings
249,012	296,111	62,551,438	61,944,406	607,032

- “Separate tracks scored highly by specific users from tracks not scored by them”
- Training set:
 - Users with items and corresponding ratings
- Test set:
 - Users with 6 items each
 - 3 rated highly, 3 not rated at all
- Task:
 - Which 3 are which
 - Binary classification (0/1 labels needed)
- Quite big dataset

Our (Zoli and the students back then) approach

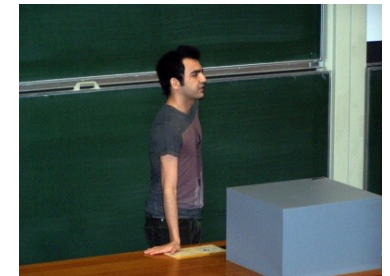
- Recall the task: 3 out of 6 songs are the current user's favorites (YES), the other 3 are random songs (NO)
- Main idea:
 - For each user, remember which albums and artists they have listened to before, from which genres
 - If a song from any of these albums, artists or even genres appears in the 6 listed, mark them as YES
 - If now you don't have exactly 3 YESes and 3 NOs, flip NOs randomly to get the 3-3 balance

Results

(approach called PIZZA)



- This simple approach
 - gives you an error rate of only 18%
 - it runs in 4 minutes
 - leaves plenty of space for improvements, e.g.
 - deal with random flips
 - establish album, artist and genre weights
 - consider similar users (collaborative filtering)
 - combine the method with many others
- After simple tweaks: down to 9.88% error rate
- For comparison, cup winner's result: 2.47%
 - but it's not a fair comparison, anyway



The winner(s)

A Linear Ensemble of Individual and Blended Models for Music Rating Prediction

Po-Lung Chen, Chen-Tse Tsai, Yao-Nan Chen, Ku-Chun Chou, Chun-Liang Li, Cheng-Hao Tsai, Kuan-Wei Wu, Yu-Cheng Chou, Chung-Yi Li, Wei-Shih Lin, Shu-Hao Yu, Rong-Bing Chiu, Chieh-Yen Lin, Chien-Chih Wang, Po-Wei Wang, Wei-Lun Su, Chen-Hung Wu, Tsung-Ting Kuo, Todd G. McKenzie, Ya-Hsuan Chang, Chun-Sung Ferng, Chia-Mau Ni, Hsuan-Tien Lin, Chih-Jen Lin and Shou-De Lin

Department of Computer Science and Information Engineering, National Taiwan University

{r99922038, r98922028, r99922008, r99922095, b97018, b97705004, b96018, b96115, b96069, b96113, b95076, b97114, b97042, d98922007, b97058, b96110, b96055, d97944007, d97041, b96025, r99922054, b96092, htlin, cjin, sdlin}@csie.ntu.edu.tw

ABSTRACT

Track 1 of KDD Cup 2011 aims at predicting the rating behavior of users in the Yahoo! Music system. At National Taiwan University, we organize a course that teams up students to work on both tracks of KDD Cup 2011. For track 1, we first tackle the problem by building variants of existing individual models, including Matrix Factorization, Restricted Boltzmann Machine, k -Nearest Neighbors, Probabilistic Latent Semantic Analysis, Probabilistic Principle Component Analysis and Supervised Regression. We then blend the individual models along with some carefully extracted features in a non-linear manner. A large linear ensemble that contains both the individual and the blended models is learned and taken through some post-processing steps to form the final solution. The four stages: individual model building, non-linear blending, linear ensemble learning and post-processing lead to a successful final solution, within which techniques on feature engineering and aggregation (blending and ensemble learning) play crucial roles. Our team is the first prize winner of both tracks of KDD Cup 2011.

1 Introduction

KDD Cup 2011 contains two tracks that are related to predicting user rating behaviors in Yahoo! Music via collaborative filtering. At National Taiwan University, similar to what was done for KDD Cup 2010 [26], we organize a course

ones per user; the training set takes the rest (earliest ones). Ratings in the test set are hidden from the contestants and the goal of track 1 is to predict those hidden ratings accurately. During the competition, a team is allowed to submit the predictions of the test set every eight hours. Then, the root-mean-squared-error (RMSE) on a fixed-but-unknown random half of the test set is reported back to the team and shown on the leaderboard (called leaderboard RMSE); the RMSE on the other half (called judge RMSE) is the final judging criterion that determines the winner of the competition. Because of the setting, using the leaderboard RMSE to tune the prediction models is both slow (once every eight hours) and dangerous (can cause overfitting). Thus, teams are expected to use the validation set, because of its relative closeness to the test set, as a local proxy of evaluating model performance. In addition to the examples, the taxonomy information between items, which can be one of genre, artist, album and track, is also provided. More details on the data set can be found in the official description [4].

The (u, i, r_{ui}, t_{ui}) parts of the data set of track 1 are similar to the one provided by the well-known Netflix competition and the judging criterion is also similar.¹ Nevertheless, three properties make the task of KDD Cup track 1 different from the task of the Netflix competition. First, the number of examples, the number of users and the number of items are much larger. Second, there is a new piece of information: the taxonomy between items. Third, there are more details



- Recipe:
 - Build a large number of recommenders
 - "Blend" them (nonlinear combination)
 - Build an ensemble of recommenders (appropriate linear combination)
 - Apply some postprocessing

Today we had:

- Introduction to recommender systems